## Transaction control language (TCL)

- Transaction : logical unit of some work that can be a multiple correlated SQL statements. Also, we call it as an atomic unit. ~~So~~ with help of these correlated SQL statements, we can perform transaction.

Using TCL, we can maintain consistancy of RDBMS.
Also, we can completely control the operations we performed around RDBMS.

## Keywords

- Save Point
- Rollback
- Locks → Put locks on table on which we are interested to retrieve / write something
  - → Read Lock
  - → Write Lock

- TCL
  - → COMMIT
  - → ROLLBACK
  - → SAVEPOINT
  - → LOCK

  } SQL

Time $T_1$

START TRANSACTION;

SELECT QUERY operation 1 (01)

INSERT QUERY 02

03 UPDATE QUERY → COMMIT; 04

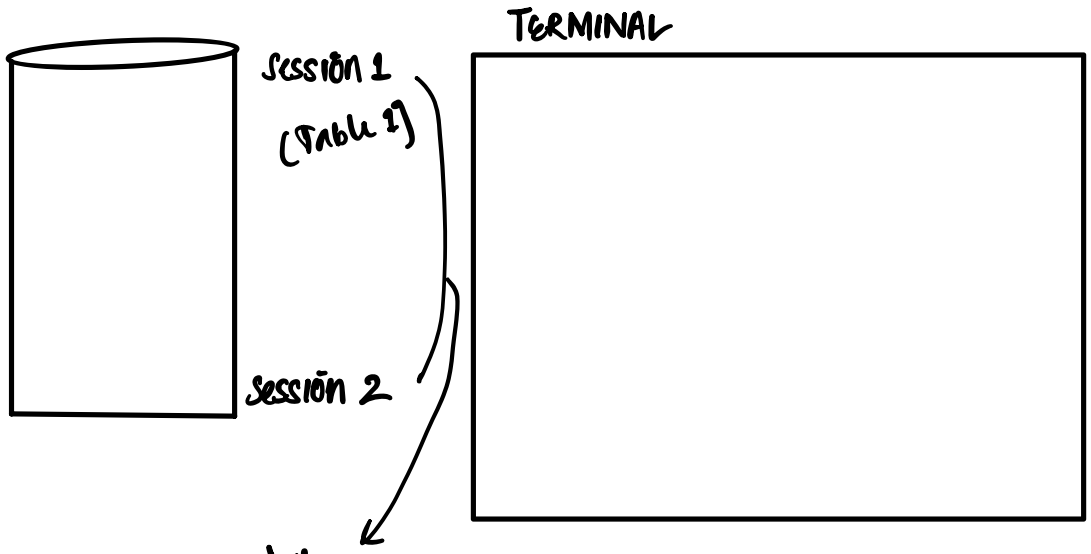
05 SAVE POINT S1;

06;

Time $T_{10}$

01
02
03 → Savepoint S1
04
05
06 → Savepoint S2
07

In case some problem happened and we are not able to execute 05 – it means we won't be able to continue with other consecutive operations → so now we ~~x~~ be able to roll back to 03




handled by different users

If lock is executed on Table 1, nobody'll be able to access Table 1.

<u>Read Lock</u>: Nobody can read on that particular period of time.

<u>Write Lock</u>: Nobody will be able to do operations — such as update anything in the data will not be done with the help of Write Lock.

# TCL commit

USE BANK;

START TRANSACTION;

INSERT INTO TRANSACTION (id, account_id, . . . ),

VALUES ('7', '2', . . . . ) ;

same order as table.

{ we have autocommit - we save everything automatically }

But what if its off?

↓

if we try to fetch same table in other session,

we won't get updated info.

↓

To avoid that ⟶ COMMIT;

# Savepoint

```
SET autocommit = OFF;

START TRANSACTION

Use Bank;

INSERT INTO Transaction ( ———— ) VALUES ( ———— );

SELECT * from Transaction

SAVEPOINT T1 ;

DELETE from Transaction where amount = 700;

ROLLBACK TO T1 ;
```

everything stored till this

Now, we'll get our deleted values.

## TCL : Lock

```
SET autocommit = OFF;
USE Bank;
SELECT * FROM transaction;
INSERT INTO TRANSACTION (——) VALUES (——);
LOCK TABLE Transaction READ;
INSERT INTO TRANSACTION (——) VALUES (——);
```

[Nobody able to write]

Error: [Table 'account' was not locked with LOCK TABLES]

(Even can't write it in new session)

↓

Session 2 will wait for Session 1 to release that lock.

Session 2 acquires lock

```
UNLOCK tables;
```

```
LOCK TABLE Transaction WRITE;
```

(SELECT query won't work)

Now, even read won't happen.

Can't write

---

```
/usr/local/mysql/bin/mysql -u root -p
```

-u → user

-p → password

```
SELECT CONNECTION_ID();
```

Just tell connection number

```
SELECT * FROM TRANSACTION;
LOCK TABLE TRANSACTION READ;
SELECT * FROM TRANSACTION;
```

INSERT INTO TRANSACTION VALUES (——);

ERROR → can't update it

[II Session → shows no reaction]

SHOW PROCESS LIST;

**WRITE LOCK**

In this session, I can do read & write

USE Bank;

LOCK TABLES TRANSACTION WRITE;

INSERT INTO TRANSACTION VALUES (——);

But in other session, we can't even read/write.

- Read lock can be shared b/w 2 sessions
- But write lock is exclusive for one session.